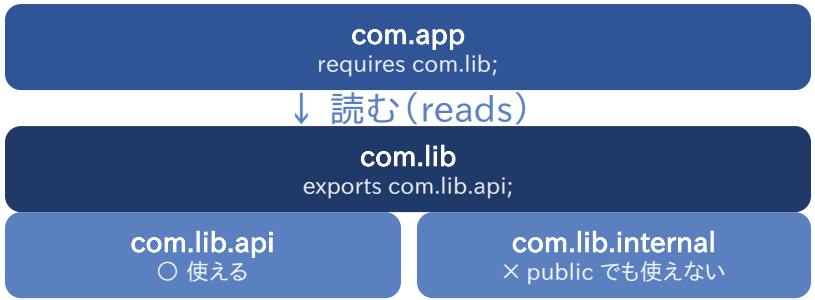
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 4 Java モジュール・システム　#10

モジュールの宣言と実行

module-info.java・requires／transitive／static・exports／opens・javac と java のオプション

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java 言語仕様 JLS 7.7、JDK 17 Tool Specifications の javac・java・jar・jlink、JEP 261）を書いた。
モジュールの構成とコマンドの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 読む（requires）× 公開する（exports）



package com.lib.internal is not visible（… which does not export it）

ソースと出力の置き方

```
src/
  com.lib/module-info.java
  com.lib/com/lib/api/Greeter.java
  com.lib/com/lib/internal/Util.java
  com.app/module-info.java
  com.app/com/app/Main.java
// javac -d out --module-source-path src -m com.app,com.lib
out/com.app/module-info.class
out/com.app/com/app/Main.class
out/com.lib/...
```

java.base はすべてのモジュールが自動で読む（requires java.base mandated）。

出典：JLS 7.7、JEP 261、javac（--module-source-path）／JDK 17 でのコンパイル結果

第1章

module-info.java

依存（requires）と公開（exports・opens）。

ディレクティブの一覧

ディレクティブ	意味	コンパイル時	実行時
requires M	M を読む	必要	必要（ないと FindException）
requires transitive M	自分を読むモジュールにも M を読ませる	必要	必要
requires static M	任意の依存	必要	なくてよい（自動では解決されない）
exports P	P の public 型を公開	効く	効く
exports P to M	M にだけ公開	効く	効く
opens P（to M）	深いリフレクションを許す	効かない	効く
open module	全パッケージを opens	中に opens は書けない	
uses・provides … with	サービス（#11）		

出典：JLS 7.7.1 Dependences・7.7.2 Exported and Opened Packages

requires transitive と requires static

```
module a.core { exports a.core; }
module b.api  { requires transitive a.core; exports b.api; }
module c.app  { requires b.api; }
// c.app から a.core.Core を使える（暗黙の読み取り）
// transitive なし → package a.core is not visible
//   (… but module c.app does not read it)
```

java.sql は java.logging・java.xml・java.transaction.xa を requires transitive。requires java.sql だけで java.util.logging も使える。

```
module use.it { requires static opt.dep; }
java -p only -m use.it/use.it.Main    // NoClassDefFoundError
java -p out  -m use.it/use.it.Main    // NoClassDefFoundError
java -p out --add-modules opt.dep -m use.it/use.it.Main // dep
```

ふつうの requires で依存がない：起動時に FindException「Module a.core not found, required by b.api」。

出典：JLS 7.7.1、JEP 261（implied readability）、java（--add-modules）／JDK 17 でのコンパイルと実行の結果

exports と opens

```
Field f = m.data.Box.class.getDeclaredField("secret");
f.setAccessible(true);
f.get(new m.data.Box());
```

m.data の宣言	結果
exports m.data;	InaccessibleObjectException
exports m.data; opens m.data;	値が取れる
open module m.data { exports m.data; }	値が取れる
open module … { opens m.data; }	'opens' only allowed in strong modules

限定公開

```
module com.lib {
  exports com.lib.api;
  exports com.lib.internal to com.app;
}
// java -d com.lib の表示：
// qualified exports com.lib.internal to com.app
```

> 同じパッケージを 2 回 exports：duplicate or conflicting exports。

出典：JLS 7.7.2、API InaccessibleObjectException／JDK 17 でのコンパイルと実行の結果

コンパイルエラーになる宣言

宣言	エラー
module my-mod { }	'{' expected（ハイフン不可）
requires java.sql; requires java.sql;	duplicate requires: java.sql
exports com.lib.api.Greeter;（クラス名）	package is empty or does not exist: com.lib.api.Greeter
exports com.lib.nothing;（クラスなし）	package is empty or does not exist: com.lib.nothing
A requires B ／ B requires A	cyclic dependence involving …
モジュールの中に package 文なしのクラス	unnamed package is not allowed in named modules
requires java.base;	エラーにならない（自動で読むのと同じ）
module m3 { }	警告だけ（should avoid terminal digits）

出典：JLS 7.7、JDK 17 でのコンパイル結果

第2章

コンパイルと実行

javac・java・jar のオプション。

javac と java

```
# まとめてコンパイル
javac -d out --module-source-path src -m com.app,com.lib
# 1 つずつコンパイル
javac -d out/com.app -p libs src/com.app/module-info.java src/com.app/com/app/Main.java
# 実行
java -p out -m com.app/com.app.Main
java --module-path out --module com.app/com.app.Main
java -p out -m com.app
# module com.app does not have a ModuleMainClass attribute,
#   use -m <module>/<main-class>
```

オプション	javac	java
-d	出力先	--describe-module
-m	コンパイルするモジュール	--module（実行する）
-p	--module-path	
-cp	--class-path（モジュールとしては扱わない）	

出典：JDK 17 Tool Specifications javac・java／JDK 17 での実行結果

調べるオプションと jar・jlink

```
java --list-modules
java.base@17.0.21
java.compiler@17.0.21 …（70 個）
java -p out --describe-module com.lib
com.lib file:///…/out/com.lib/
exports com.lib.api
requires java.base mandated
contains com.lib.internal
java -p jars --show-module-resolution -m c.app
root c.app file:///…/app.jar
c.app requires b.api file:///…/b.jar
b.api requires a.core file:///…/lib.jar
```

```
jar --create --file jars/app.jar \
    --main-class c.app.Main -C out/c.app .
jar --describe-module --file jars/app.jar
c.app jar:file:///…/app.jar!/module-info.class
requires b.api
requires java.base mandated
contains c.app
main-class c.app.Main
java -p jars -m c.app   // main-class があるので OK
```

jlink：必要なモジュールだけのランタイムを作る（--module-path・--add-modules・--output）。

出典：JDK 17 Tool Specifications java（--list-modules・--describe-module・--show-module-resolution）・jar・jlink／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
使えるか	相手のモジュールを読んでいるか（requires・transitive）＋相手が exports しているか
ディレクティブの意味	transitive は受け渡し、static はコンパイル時だけ、to は限定、opens は実行時リフレクション
エラーになる宣言	循環・重複・クラス名の exports・open module に opens・ハイフン
コマンド	-p と -m の書き方、モジュール名/メインクラス、java の -d は describe
出力先	out/モジュール名/パッケージ/クラス
実行時のエラー	依存なし → FindException、static の依存なし → NoClassDefFoundError、opens なし → InaccessibleObjectException

出典：試験内容チェックリスト 4／本資料の問題演習（全 30 問、JDK 17 で確認）